

Marketplace App: Documentación Técnica del Proyecto

Repositorio de GitHub: https://github.com/UAX-2025-26/Marketplace_Javi-Mario.git

Miembros

- Javier Yustres
- Mario Blanco

Tabla de Contenidos

1. Visión General
2. Principios Fundamentales Aplicados
3. Arquitectura de la Aplicación
4. Stack Tecnológico
5. Estructura del Proyecto
6. Guía de Puesta en Marcha
7. Decisiones Clave de Diseño

1. Visión General

Marketplace App es una aplicación de comercio electrónico para Android que sirve como un caso de estudio práctico de los principios modernos del desarrollo de aplicaciones móviles. El proyecto implementa un flujo de usuario completo: desde el registro y la autenticación, pasando por la exploración de un catálogo de productos, hasta la gestión de un carrito de compras y la persistencia de pedidos.

El enfoque principal no es solo la funcionalidad visible, sino la construcción de una base de código **robusta, mantenible y escalable**. Para ello, se ha puesto un énfasis especial en la separación de responsabilidades, la gestión del estado de la interfaz de usuario y el manejo eficiente de los datos y las operaciones en segundo plano.

Funcionalidades Implementadas

- **Autenticación de Usuarios:** Sistema de registro e inicio de sesión con correo y contraseña, utilizando **Firebase Authentication** como backend.
- **Catálogo de Productos:** Interfaz de exploración de productos optimizada para el rendimiento mediante el uso de `RecyclerView`.
- **Detalle de Producto:** Pantalla dedicada con información extendida para cada artículo.
- **Carrito de la Compra:** Gestión completa del carrito (añadir/eliminar productos) con persistencia de datos local a través de **Room**.
- **Historial de Pedidos:** Los pedidos completados se guardan en la base de datos local para su consulta posterior.

- **Navegación:** Flujo de navegación entre pantallas gestionado de forma centralizada y predecible con el **Navigation Component** de Jetpack.

2. Principios Fundamentales Aplicados

La aplicación se ha diseñado en torno a los siguientes conceptos clave:

- **Programación Dirigida por Eventos:** Es el paradigma central de la aplicación. La interacción del usuario (clics, entradas de texto) genera eventos que son capturados por *listeners*. Estos eventos no manipulan directamente la vista, sino que desencadenan acciones en la capa de lógica (`viewModel`), que a su vez actualiza el estado de la aplicación. Este flujo desacoplado es la base de una arquitectura limpia.
- **Gestión del Ciclo de Vida:** Los componentes de la aplicación, especialmente las `Activity` y `Fragment` , tienen un ciclo de vida gestionado por el sistema Android. La arquitectura se ha diseñado para ser consciente de este ciclo de vida. El uso de `viewModel` y `LiveData` asegura que los datos de la UI sobrevivan a cambios de configuración (como rotaciones) y que las actualizaciones de la UI solo ocurran cuando el componente está en un estado activo, previniendo `NullPointerException` y fugas de memoria.
- **Asincronía y Tareas en Segundo Plano:** Para garantizar una experiencia de usuario fluida (manteniendo los 60 FPS), cualquier operación potencialmente larga (acceso a base de datos, llamadas de red) se ejecuta fuera del hilo principal (UI Thread). En esta aplicación, las interacciones con la base de datos `Room` se realizan en un hilo de fondo, y los resultados se entregan al hilo principal de forma asíncrona a través de `LiveData` .

3. Arquitectura de la Aplicación

Se ha implementado el patrón de diseño **MVVM (Model-View-ViewModel)** junto con un **Patrón Repositorio**. Esta arquitectura es el estándar recomendado por Google y facilita la separación de responsabilidades de manera efectiva.

El flujo de información entre las capas es el siguiente:

- La **Vista** (los `Fragment`) captura las interacciones del usuario y las comunica al `viewModel` .
- El **viewModel** contiene la lógica de presentación. Solicita los datos que necesita al `Repositorio` .
- El **Repositorio** actúa como única fuente de verdad, obteniendo datos de la base de datos local (`Room`) o de servicios remotos (`Firebase`).
- Los datos viajan de vuelta al **viewModel**, que actualiza su estado a través de `LiveData` .
- La **Vista** observa estos `LiveData` y reacciona a los cambios, actualizando la interfaz de usuario de forma automática.

Capas de la Arquitectura

1. **Capa de UI (Vista):** Compuesta por `Fragments` . Su única responsabilidad es mostrar los datos proporcionados por el `viewModel` y capturar los eventos de entrada del usuario. No contiene ninguna lógica de negocio. Para la vinculación con los layouts XML, se utiliza `viewBinding` .
2. **Capa de viewModel:** Actúa como intermediario entre la Vista y el Repositorio. Contiene la lógica de presentación y gestiona el estado de la UI. Solicita datos al Repositorio y expone estos datos a la Vista a través de `LiveData` , de manera que la Vista puede reaccionar a los cambios de estado.
3. **Capa de Repositorio:** Implementa el patrón de *Fuente Única de Verdad (Single Source of Truth)*. Es el único punto de la aplicación que gestiona el acceso a los datos. Abstrae el origen de los mismos (la base de datos local

`Room` , un servicio de red como `Firebase`, etc.) y proporciona una API limpia al `ViewModel` para solicitarlos.

4. **Capa de Datos:** Incluye las implementaciones concretas de las fuentes de datos. En este proyecto, se compone de:
- **Room:** Para la persistencia local de datos estructurados (carrito, pedidos). Se define a través de una `Entity` (la tabla), un `DAO` (la interfaz de acceso a datos) y la clase `RoomDatabase` .
 - **Firebase:** Utilizado como un servicio de backend para la autenticación.

4. Stack Tecnológico

- **Lenguaje:** Java
- **Arquitectura Fundamental:** MVVM + Patrón Repositorio
- **Android Jetpack:**
 - **Foundation:** `AppCompat`
 - **Architecture:** `ViewModel` , `LiveData` , `Navigation Component` , `Room`
 - **UI:** `ViewBinding` , `RecyclerView` , `ConstraintLayout` , `Material Components`
- **Backend como Servicio (BaaS):** `Firebase Authentication`

5. Estructura del Proyecto

El código fuente está organizado en paquetes que reflejan la arquitectura de la aplicación:

- `com.example.marketplace.ui` : Contiene los `Fragment` y la `MainActivity` . Cada sub-paquete (`auth` , `cart` , etc.) agrupa las clases de la Vista relacionadas con una funcionalidad concreta.
- `com.example.marketplace.viewmodels` : Contiene las clases `ViewModel` , una por cada `Fragment` que requiere gestión de estado.
- `com.example.marketplace.repository` : Contiene la clase `MarketplaceRepository` , que centraliza el acceso a los datos.
- `com.example.marketplace.models` : Contiene las clases `Entity` de `Room` y otros modelos de datos (POJOs).
- `com.example.marketplace.dao` : Contiene las interfaces `DAO` (Data Access Object) para `Room`.

6. Guía de Puesta en Marcha

Prerrequisitos

- Android Studio (Recomendado: Bumblebee o superior)
- JDK 11 o superior
- Cuenta de Google para la configuración de `Firebase`

Pasos para la Configuración

1. **Clonar el Repositorio:** `git clone <URL_DEL_REPOSITORIO>`
2. **Abrir en Android Studio:** `File > Open` y selecciona la carpeta del proyecto.
3. **Configurar Firebase:**
 - Ve a la [Consola de Firebase](#) y crea un nuevo proyecto.

- Añade una aplicación Android con el nombre de paquete `com.example.marketplace`.
- Descarga el archivo `google-services.json` y colócalo en el directorio `app/` del proyecto.
- En la sección `Authentication > Sign-in method` de la consola, habilita el proveedor **Email/Password**.

4. Construir y Ejecutar:

- Permite que Gradle sincronice las dependencias (`Gradle Sync`).
- Selecciona un dispositivo o emulador y pulsa "Run".

7. Decisiones Clave de Diseño

- **Single-Activity Architecture:** Se eligió este enfoque para centralizar la gestión de la navegación y la barra de herramientas, simplificando la comunicación entre `Fragments` y proporcionando una experiencia de usuario más consistente en comparación con un modelo de múltiples actividades.
- **Exposición de LiveData Inmutable:** Los `ViewModel` utilizan internamente `MutableLiveData` para modificar el estado, pero exponen a la Vista una versión `LiveData` inmutable. Esto refuerza un flujo de datos unidireccional y garantiza que solo el `viewModel` pueda alterar el estado, lo que hace que la aplicación sea más predecible y fácil de depurar.
- **Gestión de Asincronía con Executors:** Dado que el proyecto está en Java, las operaciones de base de datos de Room, que no pueden ejecutarse en el hilo principal, se gestionan utilizando un `ExecutorService` en segundo plano. Esto asegura que la UI permanezca siempre receptiva mientras se realizan las operaciones de I/O.